Algorithms and Complexity I

Contents

1 Asymptotic notation

For two given functions $f(n) \geq 0$ and $g(n) \geq 0$:

- $f(n) = O(g(n))$ if $\exists\, c > 0,\ n_0$ such that $> f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$
- $f(n) = \Omega(g(n))$ if $\exists\, c > 0,\ n_0$ such that $> f(n) \geq c \cdot g(n) \quad \forall n \geq n_0$
- $f(n) = \Theta(g(n))$ if $\exists\, c_1, c_2 > 0,\ n_0$ such that $> c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$
- $f(n) = o(g(n))$ if $\forall c_1 > 0, \exists n_0$ such that $> f(n) \leq c_1 \cdot g(n) \quad \forall n \geq n_0$
- $f(n) = \omega(g(n))$ if $\forall c_2 > 0, \exists n_0$ such that $> f(n) \geq c_2 \cdot g(n) \quad \forall n \geq n_0$

1.1 Relationships

For any $f(n)$, $g(n)$: 1. $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$ 2. $f(n) = \Theta(g(n)) \iff f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ 3. $f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$ 4. $f(n) = o(g(n)) \iff g(n) = \omega(f(n))$ 5. $f(n) = O(g(n)) \iff$ either $f(n) = \Theta(g(n))$ or $f(n) = o(g(n))$ 6. $f(n) = \Omega(g(n)) \iff$ either $f(n) = \Theta(g(n))$ or $f(n) = \omega(g(n))$

1.2 Running time

The running time $T(I)$ of an algorithm α on input I is the time taken by α on I until α halts. Inputs are classified according to their size.

The worst-case running time $T(n)$ of algorithm α on inputs of size n is defined as $> T(n) := \max\{T(I) : I \text{ has size } n\}$

In calculating the running time of an algorithm, the RAM model of computation is used: - every primitive operation takes constant time, typically considered to be '1'

Let the pseduocode of algorithm α consist of p lines, where the execution of each line i requires time 1.

Let q_i be the number of times line i is executed on input I.

The running time of α on input I is

$$T(I) := \sum_{i=1}^{p} q_i$$

2 The knapsack problem

There are two variants of the knapsack has two variants: - The 0 - 1 knapsack problem (the 'classic' variant): - A thief robs a store and finds n items. - Item i is worth v_i euros and weighs w_i kilos. - The thief can carry at most W kilos in knapsack. - Which items can he take to maximise his profit? - The fractional knapsack problem: - same setup, but this time the thief can take fractions of items

It can be assumed that the knapsack is not big enough to carry all items, i.e., $W < w_1 + ... + w_n$, otherwise the thief would just take all items.

2.1 Greedy strategy

2.1.1 Greedy strategy for the fractional problem

The fractional knapsack problem can be easily solved using a greedy strategy: 1. Compute the value per kilo, $r_i = v_i/w_i$ for each item. 2. Sort items by value per kilo in decreasing order. 3. Take as much as possible of item with greatest 'value per kilo', then take as much as possible of item with the next greatest 'value per kilo', etc., until the total weight equals the knapsack capacity W.

For example, take $W = 10kg$ and

$$\begin{aligned} v_1 &= £4, \quad w_1 = 4\,\text{kg} \;\Rightarrow\; r_1 = 1, \\ v_2 &= £10, \quad w_2 = 2\,\text{kg} \;\Rightarrow\; r_2 = 5, \\ v_3 &= £12, \quad w_3 = 8\,\text{kg} \;\Rightarrow\; r_3 = 1.5, \\ v_4 &= £20, \quad w_4 = 5\,\text{kg} \;\Rightarrow\; r_4 = 4. \end{aligned}$$

Then, the greedy strategy takes: 1. all of item 2 (total 2 kilos), 2. all of item 4 (total 5 kilos), 3. 3 kilos of item 3,

for a total value of $10 + 20 + 3 \cdot 1.5 = 34.5$.

Running time The running time of the greedy strategy can be determined by looking at the running time of each of the steps: - Step 1 (computing r_i for each i) can be done in $\theta(n)$ - Step 2 (sorting the r_i array) can be done in $\theta(n \log n)$ - Step 3 (selecting the items) can be done in $\theta(n)$

Therefore, the worst case running time is $\theta(n \log n)$.

Corectness We have that the n items are sorted by $r_i = v_i/w_i$ in decreasing order, i.e.,

$$r_1 \geq r_2 \geq ... \geq r_n$$

Two items p and q with $r_p = r_q$ are considered to be equivalent. Hence, we may assume that:

$$r_1 > r_2 > ... > r_n$$

The total available amount of item i is w_i. Let k be such that

$$w_1 + ... + w_k \leq W \quad \text{and} \quad w_1 + ... + w_{k+1} > W$$

That is, k is the last item that can be taken completely before the knapsack capacity is exceeded. Such a k exists as we assumed the knapsack was not able to take all items.

Let $x = (x_1, ..., x_n)$ denote a solution, where $0 \leq x_i \leq w_i$ is the amount the thief takes of item i for $i = 1, ..., n$.

The total value stolen is then $\sum_{i=1}^{n} r_i x_i$.

The greedy solution x is given by

$$\begin{aligned} x_i &= w_i \quad \text{for } 1 \leq i \leq k, \\ x_{k+1} &= W - (w_1 + \cdots + w_k), \\ x_i &= 0 \quad \text{for } k+2 \leq i \leq n. \end{aligned}$$

Let $x^* = (x_1^*, \dots, x_n^*)$ be an optimal solution.

Note that we must have $x_1^* + \dots + x_n^* = W$.

Suppose $x^* \neq x$. Then: - $x_i^* < x_i$ for some $i \leq k+1$ - let $j := \min\{i : x_i^* < x_i\}$ - j is the first item that less of is taken in the optimal solution - also, $x_\ell^* > x_\ell$ for some $\ell > j$ - ℓ is the first item that more of is taken in the optimal solution

Define a new solution $y = (y_1, \dots, y_n)$ by

$$y = \begin{cases} y_j = \min\{w_j,\ x_j^* + (x_\ell^* - x_\ell)\}, \\ y_\ell = x_\ell^* - (y_j - x_j^*) \\ y_i = x_i^* \quad \text{otherwise.} \end{cases}$$

This moves as much weight as possible from item ℓ to item j: - the min ensures we don't exceed the available amount w_j - the other items remain unchanged

The total weight remains the same, W, but the value increases:

$$\sum_{i=1}^{n} r_i y_i - \sum_{i=1}^{n} r_i x_i^* = (r_j - r_\ell)(y_j - x_j^*) > 0,$$

This is a contradiction: x^* cannot be optimal if we can improve it, hence $x^* = x$

2.1.2 Greedy strategy for the 0-1 problem

The same greedy algorithm does not work for the 0-1 knapsack problem.

For instance, take $W = 6kg$, and:

$$\begin{aligned} v_1 = £3, \quad w_1 = 4\,\text{kg} \ &\Rightarrow\ r_1 = 3/4, \\ v_2 = £2, \quad w_2 = 3\,\text{kg} \ &\Rightarrow\ r_2 = 2/3, \\ v_3 = £2, \quad w_3 = 3\,\text{kg} \ &\Rightarrow\ r_3 = 2/3. \end{aligned}$$

The optimal solution is to steal objects 2 and 3 for a total of £4.

However, the greedy strategy will steal object 1 for a total of £3.

A special case of 0-1 problem Suppose that, in a 0-1 knapsack instance, the orders of the items: - when sorted by decreasing value v_i, and - when sorted by increasing weight w_i

are the same (i.e. $v_1 \geq v_2 \geq \dots \geq v_n$ and $w_1 \leq w_2 \leq \dots \leq w_n$).

This means the first item is both the most valuable and the lightest, the second is the second-most valuable and slightly heavier, etc.

In this case, applying the greedy algorithm to the problem will produce the optimal solution.

3 Greedy algorithms for (some) optimisation problems

In an optimisation problem, we are looking for a solution that optimises (i.e. maximises / minimises) some value: - in such a problem every solution has a value - an optimal solution is a solution with optimal (minimum or maximum) value. There can be many optimal solutions - the aim is to find any one of them - any algorithm for an optimisation problem goes through a sequence of steps - at each step there is a set of choices, from which one must be chosen - a greedy algorithm makes in every step the choice that looks to be the best at the moment - this is called a greedy choice or an locally optimal choice - not all optimisation problems can be solved by greedy algorithms (e.g. the 0-1 knapsack problem)

4 Divide-and-conquer algorithms

Many useful algorithms are recursive, i.e. call themselves recursively to deal with subproblems (e.g. merge sort, quick sort).

At each level of recursion, the algorithm: - divides the problem into a number of subproblems - conquers the subproblems by solving them: recursively, or directly if the subproblem sizes are small enough - combine the solutions for the subproblems into a solution for the original problem

The running times of divide-and-conquer algorithms are naturally given by recurrence relations, which express the running time on a problem of size n in terms of the running times on smaller subproblems plus the time spent dividing the problem and combining the results. For example, for merge sort:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1. \end{cases}$$

5 Cographs

5.1 Definition

Let G_1 and G_2 be two vertex-disjoint graphs.

The disjoint union $G_1 + G_2$ is the graph with: - vertex set $V(G_1) \cup V(G_2)$ and - edge set $E(G_1) \cup E(G_2)$

The join $G_1 \times G_2$ is the graph with: - vertex set $V(G_1) \cup V(G_2)$ and - edge set $E(G_1) \cup E(G_2) \cup E^*$, where E^* is the set of all edges between vertices of G_1 and vertices of G_2

A graph G is a cograph if it can be created using these rules: 1. the 1-vertex graph is a cograph 2. if G_1 and G_2 are cographs, then so is $G_1 + G_2$ 3. if G_1 and G_2 are cographs, then so is G$G_1 \times G_2$

5.2 Alternative definition

Let P_4 denote the path on 4 vertices: - a graph G contains P_4 as an induced subgraph if G can be modified to P_4 via vertex deletions - if not, then G is said to be P_4-free.

A graph G is a cograph $\iff$ G is P_4-free.

5.3 Cotree

Let G be a cograph. The cotree T_G of G is the unique decomposition tree that satisfies: 1. The root r of the tree represents the graph $G_r = G$ 2. Every leaf x of the tree represents exactly one vertex of G, and vice versa 3. Every internal node x of the tree has at least two children, it is either labeled + or $\times$, and it represents an induced subgraph G_x of G, defined as follows: - if x is a +-node, then G_x is the disjoint union of all graphs G_y where y is a child of x - if x is a $\times$-node, then G_x is the join of all graphs G_y where y is a child of x 4. Along the path from the root r to any leaf, the labels of the internal nodes alternate between + and $\times$

Any cograph G has a unique cotree. However, if we drop condition 4., there may be many trees representing G.

An example is shown below.

There is an algorithm for determining the cotree of a cograph, but it will not be included here.

5.3.1 Colouring cographs

A colouring of a graph is an assignment of colors to the vertices of the graph such that any two adjacent vertices are coloured differently.

A colouring using at most k colors is a k-colouring.

The chromatic number, χ_G of a graph G is the smallest integer k for which G has a k-colouring.

In general, this is complex, with no known polynomial-time algorithm. However, for a cograph G, its cotree T_G can be used to find its chromatic number. A bottom-up approach is followed - only process node x after first processing its children: - leaves: each leaf represents a single vertex of G. The chromatic number of a single-vertex graph is 1. - union-nodes: if x is a +-node, then χ_{G_x} is the maximum of the chromatic numbers of the graphs G_y , where y is a child of x - join-nodes: if x is a $\times$-node, then χ_{G_x} is the sum of the chromatic numbers of the graphs G_y , where y is a child of x.

This way, the last node processed is the root r, which represents the graph: $G_r = G$.

As the number of nodes of T is $O(n)$, and $O(n)$ time is spent per node, the total running time is polynomial.

6 The maximum-subarray problem

The maximum-subarray problem involves finding the maximum sum of a contigous subarray of an array A.

The brute force approach - checking all pairs of start and end indicies - is $\theta(n^2)$.

It is possible to do better, using divide-and-conquer. The idea is as follows: - split the array in half at the midpoint `mid` - any maximum subarray falls into one of three cases: - entirely in the left half, `A[low...mid]` - entirely in the right half, `A[mid...high]` - crossing the midpoint `mid` - the first two cases can be solved recursively - to solve the third case: - compute the max sum ending at `mid` from the left - compute the max sum starting at `mid + 1` from the right - add the two sums together

The running time of this is $T(n) = \theta(n \log n)$.

7 The master theorem

The master theorem is used to solve reccurence relations, and is defined as follows.

Let $f(n)$ be a function on the natural numbers, and let $T(n)$ be defined by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

for some constants $a \geq 1$ and $b > 1$. Then:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $a \cdot f(n/b) \leq c \cdot f(n)$ for some constant $c < 1$ and for sufficiently large n, then $T(n) = \Theta(f(n))$.

8 Matrix multiplication

The problem we are interested in is multiplying two square matrices, $A, B \in \mathbb{R}^{n \times n}$ to get their product, $C \in \mathbb{R}^{n \times n}$.

Using the definition of matrix multiplication, $C_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$, it can be performed in $\theta(n^3)$: - there are n^2 elements - for each element, there are n multiplications, and $n-1$ additions; $n+(n-1) = \theta(n)$

8.1 Divide-and-conquer approach

Suppose we have two $n \times n$ matrices A and B. For simplicity, assume n is a power of 2 (so we can evenly split the matrices).

Each matrix is partitioned into 4 equal-sized blocks, called block matrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Here: - Each A_{ij}, B_{ij}, C_{ij} is a $\frac{n}{2} \times \frac{n}{2}$ matrix - For example, A_{11} is the top-left quadrant of A

Then:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21}, \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21}, \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

Each of the smaller matrices can then be multiplied recursively.

The pseudocode for such an algorithm can be as follows:

```
MULT(A, B)

let C be a new $n \times n$ matrix
if n = 1 then
```

```
    c11 <- a11b11
else
    partition A, B, and C into the block matrices
    C11 <- MULT(A11, B11) + MULT(A12, B21)
    C12 <- MULT(A11, B12) + MULT(A12, B22)
    C21 <- MULT(A21, B11) + MULT(A22, B21)
    C22 <- MULT(A21, B12) + MULT(A22, B22)
return C
```

8.1.1 Time complexity

The running time of this approach satisfies $T(n) = 8T(\frac{n}{2})+\theta(n^2)$, since there are: - 8 multiplications of $\frac{n}{2} \times \frac{n}{2}$ matrices, and - 4 matrix additions, each taking time $\theta(\frac{n}{2} \cdot \frac{n}{2}) = \theta(n^2)$

Solving using the master theorem show that $T(n) = \theta(n^3)$, so this is not an improvement over using the definition.

8.2 Strassen's algorithm

Strassen's algorithm builds on this divide-and-conquer approach: 1. Divide A, B, and C into four parts as before - running time: $\theta(1)$ 2. Create 10 auxiliary matrices $S_1, \ldots, S_{10} \in \mathbb{R}^{n/2 \times n/2}$, obtained by sums or differences of $A_{11}, \ldots, B_{22}$. - running time: $\theta(n^2)$ 3. Using all the matrices available, recursively compute seven product matrices $P_1, \ldots, P7 \in \mathbb{R}^{n/2 \times n/2}$ - running time: $7T(n/2)$ 4. Compute $C_{11}, \ldots, C_{22}$ by adding/subtracting combinations of $P_1, \ldots, P7$. - running time: $\theta(n^2)$

The running time of this approach satisfies $T(n) = 7T(\frac{n}{2}) + \theta(n^2)$

Using the master theorem, we obtain:

$$T(n) = \theta(n^{\log_2 7}) = \theta(n^{2.8074})$$

9 Rod cutting problem

The rod cutting problem is as follows.

Given: - A rod of length n. - A price list p_i for $i = 1, \ldots\ n$ where p_i is the selling price of a rod of length i.

Determine the maximum revenue r_n obtained by cutting up the rod and selling the pieces.

9.1 Brute force

One way to solve this problem is by brute force - trying all possible integer combinations (we assume the rod must be cut into integer-length parts).

An integer partition of a positive integer n is a list of positive integers $a_1, \ldots, a_k$ such that $a_1 \leq a_2 \leq \ldots \leq a_k$ and $\sum_{i=1}^{k} a_i = n$.

The number of integer partitions of n increases exponentially with n.

Therefore, the brute force method has an exponential running time, and is impractical.

9.2 Optimal substructure

As a general solution,

$$r_n = \max\{p_i + r_{n-i} : 1 \leq i \leq n\}$$

Essentially, the maximum revenue from a rod of length n is obtained by choosing the best first cut length i, earning p_i from that piece, and then optimally cutting the remaining rod of length $n - i$.

Therefore, to solve the original problem of size n, we solve independent smaller problems of the same type, but of smaller sizes.

Optimal substructure means that an optimal solution to a problem can be constructed from optimal solutions to its subproblems (which may be solved independently).

9.3 Recursive top-down implementation

An algorithm that relies on optimal substructure is given in pseudocode below:

```
ROD(p, n)

if n == 0 then
    return 0
q <- -1
for i <- 1 to n do
    q <- max{q, p[i] + ROD(p, n - i)}
return q
```

This algorithm: - tries every possible first cut - solves the remaining subproblem recursively - then chooses the best option

However, this algorithm recomputes the same values many times, and therefore the time complexity is exponential.

9.4 Dynamic programming

Rather than re-solving the same subproblem repeatedly, the subproblems can be arranged so that each subproblem is solved only once, with its solution stored. If the solution is needed thereafter, it is retrieved instead of recomputed.

Dynamic programming implements this idea by using additional memory to save computation time. It is therefore an example of time-memory trade off.

A dynamic programming algorithm runs in polynomial time when both: - the number of distinct subproblems involved is polynomial - each subproblem can be solved in polynomial time

Note that for any algorithm that uses subproblems, the subproblems have to be independent. Two subproblems are independent if the solution to one subproblem does not affect the solution to another subproblem of the same problem.

Dynamic programming is effective because it utilises both: - optimal substructure (where the solution to a problem can be constructed from optimal solutions to its subproblems), and - overlapping subproblems (where the same subproblem appears in multiple larger problems)

It uses memoization, which is the process of caching the results of function calls (subproblem solutions) so that repeated calls with the same arguments can return the cached result instead of recomputing it.

9.4.1 Top-down implementation with memoization

The top-down implementation from earlier can be improved using memoization.

The procedure is still written recursively in a natural manner, but modified to save the result of each subproblem (in an array).

Them, when the procedure needs to solve a subproblem, it checks if it has previously solved this subproblem; if not, the procedure computes the value in the usual manner.

We say the recursive procedure has been memoized: it remembers what results it has computed previously.

The pseudocode for this is given below.

```
MEMOIZED-ROD(p, n)
Let r[0..n] be a new array
for i <- 0 to n do
    r[i] <- -1
return MEMOIZED-ROD-AUX(p, n, r)

MEMOIZED-ROD-AUX(p, n, r)
if r[n] >= 0 then
    return r[n]   // r[n] has been previously computed
if n == 0 then
    q <- 0   // the returned value r[0]
else
    q <- -1
    for i <- 1 to n do   // compute r[n]
        q <- max{q, p[i] + MEMOIZED-ROD-AUX(p, n-i, r)}
r[n] <- q
return q
```

9.4.2 Bottom-up implementation

Another approach to implement dynamic programming is the bottom-up method.

It depends on the idea of the 'size' of a subproblem, such that solving any particular subproblem relies on solving smaller subproblems.

The subproblems are sorted by size and solved in size order, smallest first, with the solutions saved.

Each subproblem is solved only once, and when it is first seen, all of its prerequisite subproblems have already been solved.

The pseudocode for this is given below:

```
BOTTOM-UP-ROD(p, n)
Let r[0..n] be a new array
```

```
r[0] <- 0
for j <- 1 to n do
    q <- -1
    for i <- 1 to j do
        q <- max{q, p[i] + r[j - i]}   // r[j - i] is known
    r[j] <- q
return r[n]
```

Running time For rod cutting, both the top-down and bottom-up dynamic programming approaches have a worst case running time of $\theta(n^2)$.

This is easy to see for the bottom-up approach, since it contains a (single) nested for loop.

The top-down approach solves each subproblem for sizes $0, 1, \ldots, n$ once. Therefore, to solve a problem of size n: - the for loop iterates n times - the total number of iterations is the sum $ 1 + 2 + …+ n$, an arithmetic series, giving $\theta(n^2)$

For other problems, both approaches typically have the same complexity, but: - the bottom-up approach has lower constants, due to less overhead for procedure calls - top-down may sometimes avoid solving certain subproblems entirely, if they are never needed in the recursion tree. This can make it faster in practice for some sparse problems

10 Matrix chain multiplication

Let A be a $p \times q$ matrix, and B be a $q' \times r$ matrix, i.e.,

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & \ldots & a_{1,q} \\ a_{2,1} & a_{2,2} & \ldots & a_{2,q} \\ \vdots & \vdots & \ddots & \vdots \\ a_{p,1} & a_{p,2} & \ldots & a_{p,q} \end{pmatrix}, \quad B = \begin{pmatrix} b_{1,1} & b_{1,2} & \ldots & b_{1,r} \\ b_{2,1} & b_{2,2} & \ldots & b_{2,r} \\ \vdots & \vdots & \ddots & \vdots \\ b_{q,1} & b_{q,2} & \ldots & b_{q,r} \end{pmatrix}$$

The product $C = AB$: - is not defined if $q \neq q'$ - is the $p \times r$ matrix if $q = q'$, where

$$c_{i,j} = \sum_{k=1}^{q} a_{i,k} b_{k,j}$$

This matrix has $p \cdot r$ entries, and for each entry, there are q multiplication and $q - 1$ additions $= \theta(q)$ operations per entry. - the runtime of this matrix multiplication is $\theta(pqr)$

Matrix multiplication is associative: - the multiplication ordering does not change the product - e.g, $A_1 A_2 A_3 = (A_1(A_2 A_3)) = ((A_1 A_2)A_3)$

Even though the result is the same, one order of multiplication may involve significantly fewer multiplications than another.

For example, take three matricies:

$$A_1 : 10 \times 100, \quad A_2 : 100 \times 5, \quad A_3 : 5 \times 50$$

There are two ways of parenthesizing: - $(A_1 A_2)A_3$ requires - $(10 \times 100 \times 5) + (10 \times 5 \times 50) = 7,500$ multiplications - $A_1(A_2 A_3)$ requires - $(100 \times 5 \times 50) + (10 \times 100 \times 50) = 75,000$ multiplications

The matrix chain multiplication problem is defined as follows.

Given a sequence (chain) $\langle A_1, \ldots, A_n \rangle$ of n matrices, where A_i has dimensions $p_{i-1} \times p_i$, determine the optimal parenthesization of the product $A_1 A_2 \ldots A_n$ that minimises the total number of scalar multiplications.

A product of matrices

$$A_1 A_2 \ldots A_N$$

is fully parenthesized if it is: - either a single matrix, - or the product of two fully parenthesized matrix products, surrounded by parentheses

Let $P(n)$ denote the number of ways to parenthesize a product of n matrices.

If the product is fully parenthesized, then: - it is the product of two fully parenthesized sub-products - the split of these two sub-products occurs between the kth and $(k+1)$st matrices, where $1 \leq k \leq n-1$

This provides the recurrence:

$$P(n) = \begin{cases} 1, & \text{if } n = 1 \\ \sum_{k=1}^{n-1} P(k)\, P(n-k), & \text{if } n \geq 2 \end{cases}$$

Using a brute-force method to check every possible parenthesization has an exponential running time.

10.1 Applying dynamic programming

To use dynamic programming, follow a four-step sequence: 1. Characterise the structure of an optimal solution. 2. Recursively define the value of an optimal solution. 3. Compute the value of an optimal solution. 4. Construct an optimal solution from computed information.

10.1.1 Structure of an optimal solution

To optimally parenthesize $A_i \ldots A_j$, suppose that we split the product between A_k and A_{k+1}: - both parenthesizations of $A_i \ldots A_k$ and $A_{k+1} \ldots A_j$ must be optimal

The cost of the optimal solution for $A_i \ldots A_j$ is the sum of: - the cost to optimally parenthesize $A_i \ldots A_k$ - the cost to optimally parenthesize $A_{k+1} \ldots A_j$ - the cost of multiplying them together

10.1.2 Recursive definition of the value of an optimal solution

Let $m[i, j]$ be the minimum number of multiplications required to compute $A_i \ldots A_j$.

The goal is to compute $m[1, n]$.

$m[1, n]$ can be computed recursively, for $i \leq j$:

$$m[i, j] = \begin{cases} 0, & \text{if } i = j, \\ \min\limits_{i \leq k < j} \left\{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \right\}, & \text{if } i < j. \end{cases}$$

There are two cases: - the base case: if $i = j$, then $m[i, j] = 0$, since a single matrix needs no multiplication - the recursive case: if $i < j$, then: - check all possible places to split - use the minimum of all of these

10.1.3 Computing optimal costs

There are only a few distinct subproblems: - one for every pair of i and j, so there are $\theta(n^2)$ subproblems - there are many overlapping subproblems

A bottom-up approach can be used: - computing $m[i, j]$ for a product of $j - i + 1$ matrices only uses values for products of fewer than $j - i + 1$ matrices. - order matrices by their chain length $\ell = j - i + 1$ - this is just the number of matricies being multiplied in the subproblem - store the answers in a table $m[1 \ldots n, 1 \ldots n]$ - starting from the shortest chain length (the smallest number of matricies in the product), calculcate $m[i, j]$ - first fill $m[i, i] = 0$ - then compute chains of length 2, then length 3 etc. until length n - for each chain, try every possible split and choose the cheapest

Pseudocode for this is given below:

```
MATRIX-CHAIN-ORDER(p)
Let m[1..n, 1..n] be a new table
for i <- 1 to n do
    m[i, i] <- 0
for l <- 2 to n do
    for i <- 1 to n - l + 1 do
        j <- i + l - 1
        m[i, j] <- inf
        for k <- i to j - 1 do
            m[i, j] <- min{m[i, j], m[i, k] + m[k + 1, j] + pi-1pkpj}
return m
```

So far, we have determined the minimum number of multiplications, but not where the parenthesis should be placed to achieve it.

However, it is easy to modify the solution to keep track of where to place the parenthesis: - for every i and j, it is enough to remember the cut point k, where we cut the product $A_i \ldots A_j$ into $A_i \ldots A_k$ and $A_{k+1} \ldots A_j$ - an additional array $s[i, j]$ can be used to keep track of all those ks

```
MATRIX-CHAIN-ORDER(p)
Let m[1..n, 1..n] and s[1..n - 1, 2..n] be new tables
for i <- 1 to n do
    m[i, i] <- 0
for l <- 2 to n do
    for i <- 1 to n - l + 1 do
        j <- i + l - 1
        m[i, j] <- inf
        for k <- i to j - 1 do
            q <- min{m[i, j], m[i, k] + m[k + 1, j] + pi-1pkpj}
            if q < m[i, j] then
                m[i, j] <- q
                s[i, j] <- k
return m
```

10.1.4 Constructing an optimal solution

The table $s[1..n-1, 2..n]$ hold the information to print out the right parenthesization.

```
Algorithm PRINT-PAR(s, i, j)
if i == j then
    print "Ai
else
    print "("
    PRINT-PAR(s, i, s[i, j])
    PRINT-PAR(s, s[i, j] + 1, j)
    print ")"
```

This part has running time $O(n)$.

Running time The main algorithm contains three nested for loops, therefore the running time of the algorithm is $O(n^3)$.

It can further be shown that the running time is actually $\theta(n^3)$.

11 Longest common subsequence

The longest common subsequence problem is as follows.

Given two sequences, find the longest sequence of characters (longest common subsequence) that: - appears in both sequences - appears in the same order - is not necessarily contiguously (i.e., characters don't have to be next to each other)

For example, for the strings `S1` and `S2` below, `S3` is the longest common subsequence:

Formally, given a sequence $X = \langle x_1, \dots, x_m \rangle$, another sequence $Z = \langle z_1, \dots, z_k \rangle$ is a subsequence of X if there exist indices $i_1 < i_2 < \cdots < i_k$ such that
$z_1 = x_{i_1},\ z_2 = x_{i_2},\ \dots,\ z_k = x_{i_k}$. - the elements do not have to be consecutive — only the order must be preserved

A common subsequence of X and Y is a subsequence of both X and Y.

11.1 Brute-force

A possible brute-force approach is to: - enumerate all subsequences Z of X - check whether Z is also a subsequence of Y - keep track of the longest subsequence that appears in both

If X has m indicies, there are 2^m subsets - therefore, the brute force approach has exponential running time

11.2 Dynamic programming

Let $X = \langle x_1, x_2, \dots, x_n \rangle$, and let the ith prefix of X be $X_i = \langle x_1, x_2, \dots, x_i \rangle$.

11.2.1 Optimal substructure

Let $Z = \langle z_1, \dots, z_k \rangle$ be an LCS of $X = \langle x_1, \dots, x_m \rangle$ and $Y = \langle y_1, \dots, y_n \rangle$.

1. If $x_m = y_n$, then:

- $z_k = x_m = y_n$
- Z_{k-1} is an LCS of X_{m-1} and Y_{n-1}

2. If $x_m \neq y_n$, then:
   - $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y
3. If $x_m \neq y_n$, then:
   - $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1}

In words, this means: - look at the last character of X and Y - if the last characters match: - this character must be the last character in the LCS, because if it weren't used, we could append it and get a longer common subsequence (contradiction) - everything before it, Z_{k-1}, must be an LCS of the prefixes X_{m-1} and Y_{n-1} - if the last characters don't match: - at least one of them is not part of the LCS - if the LCS does not end with the last character of X, then discarding this character X_m does not affect the LCS - therefore, Z is an LCS of X_{m-1} and Y - if the LCS does not end with the last character of Y, then discarding this character Y_n does not affect the LCS - therefore, Z is an LCS of X and Y_{n-1}

Therefore, there is an optimal substructure: - an LCS of X and Y contains within it an LCS of some prefixes of X and Y

One or two subproblems have to be examined each time: 1. if $x_m = y_n$ we must find an LCS of X_{m-1} and Y_{n-1} 2. if $x_m \neq y_n$ we have to find: - an LCS of X_{m-1} and Y - an LCS of X and Y_{n-1} - keep whichever is longer

There is also the overlapping subproblems property: - we may need to find LCS's of $\{X_{m-1}, Y\}$ and of $\{X, Y_{n-1}\}$ - each of these subproblems has the subproblem of finding an LCS of $\{X_{m-1}, Y_{n-1}\}$ - therefore, many other subproblems share common subproblems

11.2.2 Recursive solution

Let $c[i, j]$ be the length of an LCS of the prefixes X_i and Y_j.

Therefore,

$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max\{c[i, j-1],\, c[i-1, j]\} & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

Unlike the rod cutting and the matrix chain multiplication problems, here we rule out some subproblems based on the conditions of the problem (i.e. $x_i = y_j$ or $x_i \neq y_j$).

Used directly, this recursive equation gives an exponential time recursive algorithm. However: - every recursive call is of the form c$[i, j]$ - so a subproblem is identified by the pair (i, j), where $i \in \{0, 1, \ldots, m\}$ and $j \in \{0, 1, \ldots, n\}$ - so the number of distinct subproblems $= (m+1)(n+1) = \theta(mn)$

Therefore, using dynamic programming to only solve each subproblem once and store the result for reuse if needed, reduces the algorithm to polynomial time, more specifically, $\theta(mn)$.

11.2.3 Computing the length of an LCS

A bottom-up approach can be used: - create a length table $c[0 \ldots m, 0 \ldots n]$ - $c[i, j]$ is length of the LCS of prefixes X_i and Y_j - create a direction table $b[1 \ldots m, 1 \ldots n]$ - $b[i, j]$ is a pointer that tells us how the optimal value in $c[i, j]$ was obtained - this table is used later to reconstruct the actual

LCS - initialise the base cases: - for all i, set $c[i, 0] = 0$ - for all j, set $c[0, j] = 0$ - fill the tables in row by row - once the table is complete, the length of the LCS is stored in: $c[m, n]$

Pseudocode for this is given below:

```
LCS(X, Y)
Let b[1..m, 1..n] and c[0..m, 0..n] be new tables
for i <- 0 to m do c[i, 0] <- 0
for j <- 0 to n do c[0, j] <- 0
for i <- 1 to m do
    for j <- 1 to n do
        if xi == yj then
            c[i, j] <- c[i - 1, j - 1] + 1
            b[i, j] <- "UL"  // upper-left subproblem
        else if c[i - 1, j] >= c[i, j - 1] then
            c[i, j] <- c[i - 1, j]
            b[i, j] <- "U"  // upper subproblem
        else
            c[i, j] <- c[i, j - 1]
            b[i, j] <- "<-"  // left subproblem
return c and b
```

11.2.4 Constructing an LCS

To recover the actual subsequence (not just its length): - start at cell (m, n) - follow the pointers in b: - whenever we see a diagonal pointer (`UL`): - output the corresponding character - move to $(i-1, j-1)$ - if the pointer is up (`U`), move to $(i-1, j)$ - if the pointer is left (`<-`), move to $(i, j-1)$ - stop when you reach the top row or leftmost column - the characters collected (in reverse order) form the LCS

```
PRINT-LCS(b, X, i, j)
if i == 0 or j == 0 then
    return ""
if b[i, j] == " UL " then
    PRINT-LCS(b, X, i - 1, j - 1)
    print xi
else if b[i, j] == " U " then
    PRINT-LCS(b, X, i - 1, j)
else
    PRINT-LCS(b, X, i, j - 1)
```

Outputting the LCS is $O(m + n)$, as the total number of steps is at most $m + n$.

An example, combining both tables, is given below:

12 Matching in graphs

Let $G = (V, E)$ be an undirected graph.

When modeling a real-world application, usually: - vertices $v \in V$ represent some objects of interest, e.g. cities on a map - edges $e \in E$ represent relations between pairs of vertices, e.g. roads between

cities

However, often the edges become the interesting objects to study - an edge unites or matches two vertices

A matching $M \subseteq E$ in G is an independent (non-conflicting) set of edges, i.e. a set of edges where no two edges share a common vertex. No object is used twice; there are no conflicts, as each object is matched with at most one partner.

Matchings are useful as many real-world problems are about making non-conflicting pairings. - for example, suppose there is a graph where each vertex represents a football team, and each edge represents a potential game between two teams - if a number of football games are to be scheduled at the same time, a matching would be required, as a team can only play one other team at a time

A bipartite graph is a graph where the verticies can be split into two disjount groups, such that all edges go between the two groups, and never within the same group. Such a graph can be two-coloured.

Formally, a graph $G = (V, E)$ is bipartite if its vertex set V can be partitioned into two sets, $V = X \cup Y$ such that - $X \cap Y = \emptyset$ (they don't overlap) - every edge connects a vertex in X to a vertex in Y - no edge connects two vertices both in X - no edge connects two vertices both in Y

Many assignment problems can be formulated as finding a special matching in a bipartite graph $G = (X \cup Y, E)$. - for example, verticies in X are workers, and verticies in Y are projects to be assigned to workers; each edge represents a job a worker is able to do

A matching M is called: - maximal, if M is not a proper subset of any other matching M' in G - maximum, if no other matching M' in G contains a strictly greater number of edges - perfect, if every vertex of G is an endpoint of an edge in M

The matching number ν_G of G is the size of a maximum matching in G.

Note: - Every maximum matching is also maximal - A maximal matching is not necessarily maximum - A perfect matching is always maximum

12.1 Alternating paths and cycles

Let $G = (V, E)$ be a graph and M be a matching in G.

Let $P = (v_0, v_1, \dots, v_k)$ be a simple path with edges $e_1 = v_0v_1, e_2 = v_1v_2, \dots, e_k = v_{k-1}v_k$.
Then: - P is an M-alternating path if the edges $e_1, e_2, \dots, e_k$ are alternately in M and in $E \setminus M$. - P is an M-augmenting path if P is M-alternating and neither v_0 nor v_k is covered by M - this requires $M = \{e_2, e_4, \dots, e_{k-1}\}$ - therefore, k is odd in an augmenting path

Similarly, a cycle C in G is an M-alternating cycle if its edges alternately belong to M and $E \setminus M$.

A vertex u is matched by M if it is the end-vertex of an edge in M; otherwise u is unmatched by M.

For two sets A and B, the symmetric difference is the set $A \otimes B = (A \setminus B) \cup (B \setminus A)$. - for example:
Let $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$
Then:
$A \setminus B = \{1, 2\}$

$B \setminus A = \{5, 6\}$
so $A \otimes B = \{1, 2, 5, 6\}$

Lemma Let G be a graph with a matching M and an M-alternating path $P = (v_0, v_1, ..., v_k)$. If each of the endpoints v_0, v_k of P is: - either unmatched by M,
- or matched by $M \cap P$ (matched using edges inside the path)

then $M \otimes P$ is another matching in G.

Corollary If P is an M-augmenting path, then in the new matching $M \otimes P$:

- $|M \otimes P| = |M| + 1$,
- all the matched vertices in M remain matched,
- two additional vertices v_0, v_k become matched in $M \otimes P$.
  - this is called an augmentation of M: replacing M with $M \otimes P$.

Lemma Let $G = (V, E)$ be an undirected graph, and let M and $M*$ be two matchings in G. Then the subgraph$H = (V, M \otimes M*)$ of G is the disjoint union of: - isolated vertices, - alternating cycles with respect to both M and $M*$, - alternating paths with respect to both M and $M*$.

Lemma Let $G = (V, E)$ be a graph. The following are equivalent: 1. M is a maximum matching 2. G has no M-augmenting path

Proof First, it must be proved that if M is a maximum matching, then there is no M-augmenting path: - Let G be a graph with a maximum matching M and an M-augmenting path P. - Then $|M \otimes P| = |M| + 1$, i .e. the matching $M \otimes P$ has more edges than M. - hence, M is not a maximum matching - this is a contradiction, therefore, if M is a maximum matching, then there is no M-augmenting path

Next, it must be proved that if there exists no M-augmenting path P, then M is a maximum matching: - Suppose M is not a maximum matching - Let M' be a maximum matching - The subgraph $H = (V, M \otimes M')$ contains only alternating paths and cycles (with respect to M and M') - Each alternating cycle C has an even number of edges - therefore, C has the same number of edges in M and M' - Since $|M'| > |M|$, there must be an odd path P in H, in which the first and last edges belong to M' - P is an M-augmenting path - this is a contradiction, therefore, M is a maximum matching

Corollary Let $G = (V, E)$ be a graph, and let M and $M*$ be two matchings in G such that $|M*| = |M| + k$ for some $k \geq 1$. Then G has at least k pairwise vertex-disjoint M-augmenting paths.

Proof

- The subgraph $H = (V, M \otimes M*)$ consists of alternating paths and alternating cycles with respect to both M and $M*$ (and possibly isolated vertices).
- The symmetric difference $M \otimes M*$ contains exactly k more edges from $M*$ than from M.
- Therefore, H contains k pairwise vertex-disjoint odd-length paths that start and end with edges of $M*$.
- These paths are M-augmenting paths.

The main takeaway from the lemmas above is that if a matching is not maximum, then there exists an augmenting path, and if you flip along that path, the matching gets bigger by 1.

Therefore, there is an algorithmic method for solving the matching problem: 1. Let $M = \emptyset$ 2. Check if there is an M-augmenting path P 3. If P exists, then $M := M \otimes P$ and go to Step 2. Otherwise output M.

This requires an algorithm to determine if there is an augmenting path P: - A vertex u is M-reachable from v if there is an M-alternating path from v to u: - therefore, use a search algorithm to find all M-reachable vertices from v - if we find an unmatched M-reachable vertex u then there is an M-augmenting path from v to u - otherwise, there is no M-augmenting path from v, so try other verticies

The idea for the search algorithm is: - grow a search tree T rooted at v - each path from v to any vertex u in the tree is alternating - every vertex that is visited is labeled odd/even - if vertex u is odd/even, the path from v in T is odd/even - if an unmatched vertex is labeled odd, an augmenting path has been found

Note that this only works for bipartite graphs, since they do not have odd-cycles.

The algorithm can then work as follows: - The algorithm maintains a LIST of labeled vertices. Initially, LIST = $\{v\}$, label(v) = "even". - It iteratively removes a vertex from LIST and continues searching. - To examine an even vertex u: - Scan its neighbors $N(u)$. - If $w \in N(u)$ is unlabeled: - assign label(w) = "odd", - add w to LIST. - If w is unmatched, then an augmenting path has been found and the algorihtm can stop - To examine an odd vertex u (which has been matched): - Explore the unique edge $ux \in M$. - If x is unlabeled: - assign label(x) = "even", - add x to LIST.

The algorithm terminates if: - either LIST is empty
- there is no augmenting path from v,
- or we assign the label "*odd*" to an unmatched vertex
- an augmenting path has been found.

12.1.1 Running time

Since we only explore each edge once, the algorithm for finding an augmenting path runs in $O(|E|)$ time.

Each occasion results in a matching that has one more edge than the previous matching. The largest possible matching has $\frac{|V|}{2}$ verticies, so the augmenting path algorithm is run at most $\frac{|V|}{2}$ times.

Therefore, the total running time of the algorithm for finding a maximum matching in a bipartite graph is $O(|E||V|)$ time.